

Descrizione generale del sistema

Il sistema di interesse scelto è un sistema di noleggio veicoli che permetta di gestire in modo semplice, ordinato e completo tutte le operazioni tipiche di un servizio di noleggio.

L'applicazione permette di registrare i clienti, gestire un parco veicoli composto da automobili e motocicli, creare e monitorare i noleggi, registrare i pagamenti e tenere traccia delle manutenzioni effettuate sui veicoli.

Presentazione del dominio scelto

Nel seguente dominio di interesse abbiamo individuato :

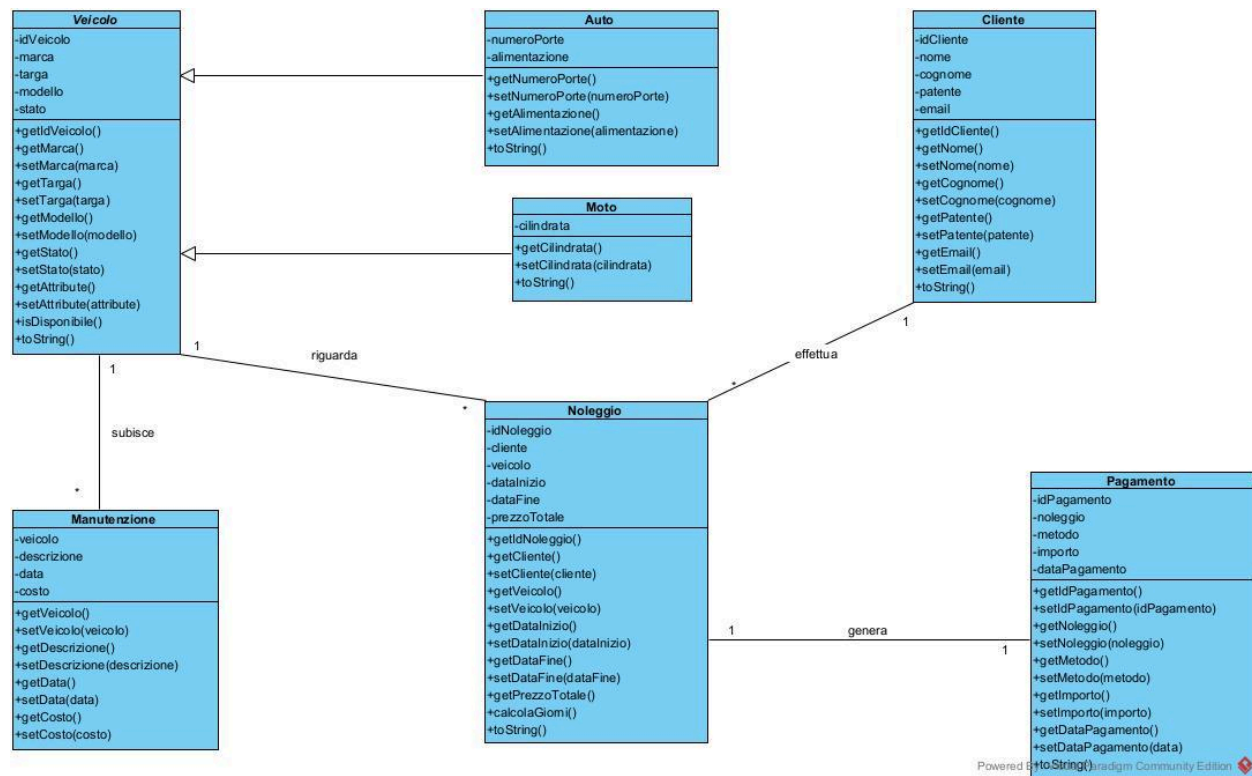
- entità (Cliente, Veicolo, Noleggio, Pagamento, Manutenzione)
- relazioni (un cliente effettua noleggi, un veicolo può essere noleggiato più volte, ogni noleggio genera un pagamento, ecc.)
- ereditarietà reale (Auto e Moto sono specializzazioni di Veicolo)
- logiche di business (verifica disponibilità, calcolo giorni, gestione stato del veicolo)
- processi completi che vanno dalla registrazione del cliente fino alla chiusura del noleggio

Relazioni tra le classi

Le relazioni sono state modellate in modo da riflettere fedelmente il comportamento reale del dominio:

- Cliente effettua Noleggio ($1 \rightarrow *$) : un cliente può effettuare più noleggi , ogni noleggio è effettuato da un solo cliente
- Noleggio riguarda Veicolo ($1 \rightarrow *$) : Un noleggio può riguardare più veicoli , ogni veicolo è inserito in un solo noleggio
- Noleggio genera Pagamento ($1 \rightarrow 1$) : ogni pagamento è generato da un noleggio è ogni noleggio genera un pagamento
- Veicolo subisce Manutenzione ($1 \rightarrow *$) : un veicolo può subire più manutenzioni ogni manutenzione può essere fatta su un veicolo
- Veicolo è superclasse di Auto e Moto (ereditarietà)

Class Diagram



Scelte progettuali

Tutti gli attributi id sono gli unici a non avere un metodo setter dedicato onde evitare che possano essere modificati in qualsiasi punto del programma , garantendone l'unicità. Per facilitare l'implementazione del sistema della realtà di interesse da noi scelto abbiamo sviluppato un sistema che permetta di noleggiare esclusivamente automobili e motocicli. Abbiamo inoltre pensato che un veicolo potesse subire più manutenzioni e che quest'ultima non possa essere collegata a più veicolo in quanto questi potrebbero essere soggetti a manutenzioni diverse. Ogni classe che potrebbe essere utile da stampare ha un override del metodo `toString()` , l'attributo `prezzoTotale` della classe `noleggio` non ha un metodo setter in quanto il prezzo del noleggio viene definito in base alla durata dei giorni in cui il veicolo è stato noleggiato , calcolato grazie all'implementazione dell' apposito metodo nella medesima classe. Il metodo `isDisponibile` ci permette di verificare in quali tra i seguenti stati si trova il veicolo : “disponibile” , “noleggiato” e “in_manutenzione” per verificare la fattibilità o meno del noleggio.